

◆ Documentação técnica · Palmas-TO

APPDELYVERY

Arquitetura, progresso, plano de entrega, polimento do front e estratégia de captação — o mapa completo do projeto.

Impulso Digital · junho / 2026

SUMÁRIO

- 01** Arquitetura do Sistema
- 02** Checklist de Progresso
- 03** Plano de Entrega
- 04** Polimento do Front, Testes e Vitrine
- 05** Estratégia de Captação

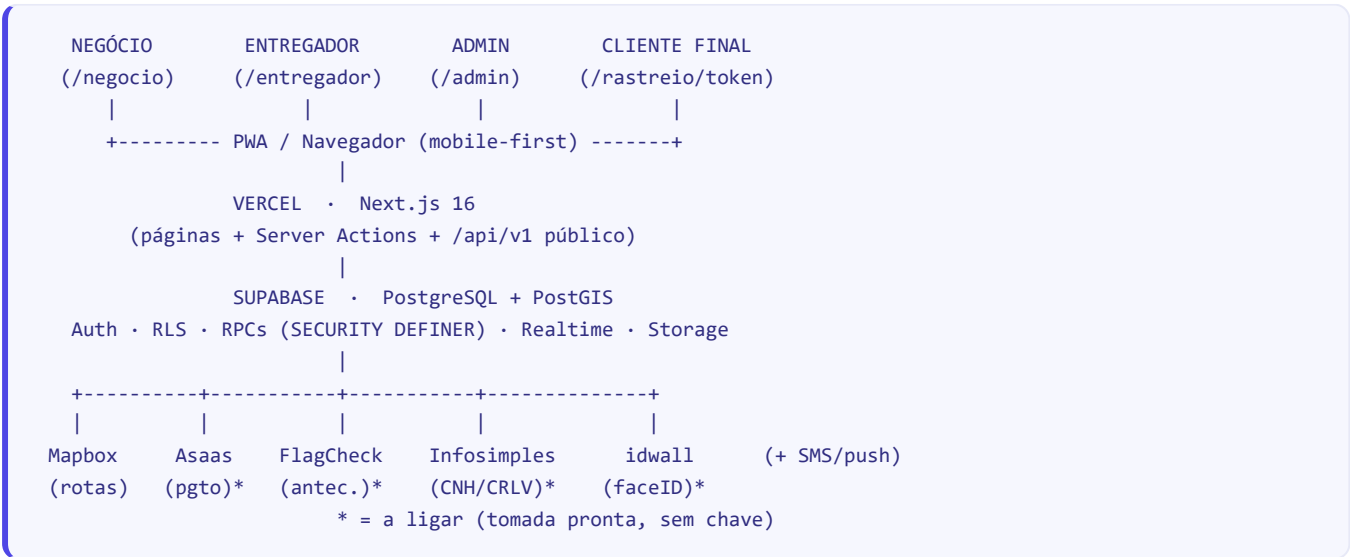
Arquitetura do Sistema

Como a plataforma é montada por dentro: stack, camadas, fluxos, banco, segurança e integrações. Estado real em 08/06/2026 (auditoria de código+banco).

1. Stack (o que usamos)

CAMADA	TECNOLOGIA	PAPEL
App + Site	Next.js 16 (App Router) + React 19 + TypeScript + Tailwind v4	As 4 telas + vitrine, um app só por papel; PWA instalável
Banco + Auth	Supabase (PostgreSQL + PostGIS)	Dados, login, regras de acesso (RLS), tempo real, arquivos
Lógica de negócio	Server Actions + RPCs SECURITY DEFINER no Postgres	Cérebro no banco (despacho, preço, guards)
Hospedagem	Vercel	Deploy contínuo (push na <code>main</code> → no ar)
Mapas/rotas	Mapbox (Directions + Geocoding)	Rota real pelas ruas, endereço, distância
Pagamento	Asaas (Pix/split)	Carteira, split 80/20, repasse — <i>a ligar</i>
Verificação	FlagCheck + Infosimples + idwall	Antecedentes + CNH/CRLV + biometria — <i>a ligar</i>
Notificação	SMS/WhatsApp + Push	Avisos — <i>a definir/construir</i>

2. Camadas (visão de cima)



3. Os 4 atores (mesmo sistema, rota por papel)

O login lê `profiles.role` e redireciona: `estabelecimento` → `/negocio`, `entregador` → `/entregador`, `admin|operador` → `/admin`. O cliente final não tem login — entra por **link com token** (`/rastreo/[token]`).

- **Negócio** — cria entrega (3 veículos + preço + endereço), acompanha, histórico, carteira, relatórios, API.
- **Entregador** — cadastro→verificação→oferta→coleta→rota→entrega; ganhos; perfil.
- **Admin** — despacho ao vivo, corridas, aprovação de entregador (PIN), financeiro, config, comunicados.
- **Cliente final** — rastreo por link, GPS ao vivo, chat, código de entrega.

4. Modelo de dados (núcleo)

`profiles` (papel) → `estabelecimentos` / `entregadores` → **`pedidos`** (centro de tudo) → `ofertas`, `rastreios` (GPS), `comprovaes` (foto/assinatura), `pagamentos`, `avaliacoes`, `carteira_transacoes`, `mensagens` (chat 3-pontas), `cancelamentos`, `disputas`, `comunicados`, `chaves_api`, `config` (preços/take/PIN/raio singleton). **27 migrations**, RLS em todas, PostGIS para geolocalização.

5. Fluxos principais

A) Entrega (manual): `lojista cria` → `status 'buscando'` → `trigger` dispara o motor → `ofertar_proximo` (ranking `distancia*nota`, timer 30s) → `entregador aceita` (atômico) → `coleta` (foto + geofence) → `rota` (GPS ao vivo via Realtime) → `entrega` (foto + assinatura + código 4 dígs) → `'entregue' + split 80/20`.

B) Entrega (via API — clientes grandes): `sistema do lojista` → `POST /api/v1/pedidos` (chave) → `criar_pedido_via_api` (preço calculado no banco) → mesmo motor de oferta → `webhook HMAC` devolve `status`.

C) Rastreo do cliente: `link com token` → `get_rastreo_publico` (`status + entregador + posição`) → GPS ao vivo (Realtime) → chat por token.

D) Verificação do entregador: `cadastro` → `iniciarVerificacao` (`FlagCheck + Infosimples + idwall`, server-side) → grava em `verificacoes` (só-admin, LGPD) → admin aprova com PIN.

E) Pagamento (a construir): `carteira pré-paga` (recarga Pix via Asaas) → `desconta` na criação → `split 80/20` na conclusão → repasse D+1 pro entregador.

6. Motor de despacho (oferta dirigida — modelo 99)

Cérebro em funções Postgres + PostGIS. `ofertar_proximo` ranqueia por **distância real × nota** (placar adaptativo: quem entrega bem sobe). Oferta dirigida com cronômetro (~30s); recusou/expirou → cascata pro próximo. Aceite **atômico** (nunca corrida dupla). `processar_ofertas` roda por **cron** (expira/reactiva). Parametrizável por cidade (raio, pesos, preço) — base pra escalar pra outras praças.

7. Segurança & LGPD

- **RLS** em todas as tabelas; helpers `is_admin()` / `auth_role()` SECURITY DEFINER (sem recursão).

- **Guards (triggers):** anti self-promote a admin; status crítico (coletado/entregue) só com evidência (foto/assinatura); entregador não se auto-aprova.
- **Antecedentes = dado sensível:** tabela `verificacoes` só-admin, nunca exposto ao negócio/cliente nem em URL.
- **Acesso público por token** (rastreo/chat) via SECURITY DEFINER que devolve só o necessário.
- **API:** chave com hash SHA-256; webhook assinado (HMAC); preço sempre recalculado no servidor.

8. Integrações (status real)

INTEGRAÇÃO	STATUS	FALTA
Mapbox	Ligada	—
Supabase	Ligada	—
FlagCheck / Infosimples / idwall	Tomada no-op	chaves no CNPJ do dono + <code>SUPABASE_SERVICE_ROLE_KEY</code>
Asaas	Esqueleto (webhook 501)	construir carteira/split/repassse + chave
SMS/WhatsApp + Push	Ausente	definir provedor + construir

9. Deploy & ambientes

Repo `github.com/AppDelyvery/appdelivery` (conta gh dedicada **AppDelyvery**). Push na `main` → **Vercel** deploys (appdelivery-psi.vercel.app). Supabase projeto `cqmxjzrukbagvkznrunt`. Segredos em env var (Vercel + `.env.local`); contas externas no **CNPJ do dono**. Regra dura: `tsc --noEmit` + `next build` + migration aplicada antes do deploy.

10. Evolução planejada (Fase 2)

Apps nativos (Expo/Play+App Store) reaproveitando a mesma API; Van intercidade; conectores Bling/Tiny/PDV; camada de IA (previsão de demanda, ETA preditivo, antifraude, score) sobre o motor determinístico atual; multi-praça (a parametrização por cidade já é a base).

Checklist de Progresso

Estudo de estado real do projeto, levantado por auditoria do **código e do banco** (não dos docs antigos) em **08/06/2026**. Serve pra medir progresso e pra separar, no doc de vendas, o que é **setup (R\$15k)** do que é **evolução (Fase 2, à parte)**.

Legenda: [x] pronto e provado · [~] construído mas falta ativar/ligar · [] a fazer

Leitura rápida: a plataforma (telas, banco, motor, API, GPS, deploy) está praticamente toda **pronta e no ar**. O que falta pra "operar de verdade" não é tela — é **ligar o dinheiro (Asaas)** e a **verificação (chaves no CNPJ)**, mais SMS/push e o jurídico. Estimativa: **~80% do setup pronto**; os ~20% restantes são integrações externas + polimento.

A. PRONTO E NO AR (provado na fonte)

Fundação

- ✓ Stack Next 16 + Supabase + Vercel + Tailwind, deploy contínuo (push → Vercel)
- ✓ No ar: **appdelivery-psi.vercel.app** (site, login, app, rastreo)
- ✓ Auth real (cadastro/login) + redirect por papel (negócio/entregador/admin)
- ✓ PWA instalável (manifest, ícones) — abre no `/login` e roteia por papel
- ✓ SEO/AEO (metadata, llms.txt, robots, sitemap)
- ✓ Design system (Inter, índigo, lcons.tsx SVG, zero emoji), responsivo mobile

Banco (27 migrations aplicadas — 0001 a 0027)

- ✓ ~18 tabelas (profiles, estabelecimentos, entregadores, pedidos, ofertas, comprovantes, mensagens, etc)
- ✓ ~30 RPCs/funções + PostGIS (geolocalização)
- ✓ RLS em todas as tabelas + guards de segurança (anti self-promote admin, status só com evidência, entregador não se auto-aprova)
- ✓ Auditoria adversarial de RLS (2 furos achados e fechados)

3 personas + cliente (todas no ar)

- ✓ **Negócio** (8 telas): novo pedido + mapa, histórico, perfil, comunicados, relatórios/CSV, integração API
- ✓ **Entregador** (5 telas): cadastro→verificação→oferta→coleta→rota→entrega, ganhos, perfil, comunicados
- ✓ **Admin** (12 telas): dashboard, despacho ao vivo, corridas+comprovante, entregadores+aprovação(PIN), negócios, financeiro, rankings, avaliações, mensagens, disputas, comunicados, config
- ✓ **Cliente final**: rastreo por link (sem login) + GPS ao vivo + chat

Motor e recursos

- ✓ Algoritmo de oferta dirigida (modelo 99: ranking distância × nota, timer 30s, cascata, aceite atômico) + cron
- ✓ GPS ao vivo (Supabase Realtime broadcast)
- ✓ Preço por veículo (moto/carro/van) com match exato + config editável no admin

- ☒ Comprovação forte (foto + assinatura + código de 4 dígitos) + geofence anti-fraude
- ☒ Cancelamento com motivo (entregador e lojista), chat 3-pontas, proteção de carga (teto config)
- ☒ **API de integração** (criar pedido + cotação + webhook HMAC) — provada no ar (modelo Bee→Drogasil)
- ☒ Mapbox real (rotas pelas ruas + geocoding + tradutor de endereço de Palmas/ARSE)

B. CONSTRUÍDO, MAS FALTA ATIVAR (precisa conta/chave no CNPJ do dono)

O código está pronto como "tomada no-op" — liga quando a chave entra. Hoje não trava o fluxo (cai em modo manual).

- [~] **Verificação — Antecedentes (FlagCheck)** — falta `FLAGCHECK_API_KEY`
- [~] **Verificação — CNH/CRLV (Infosimples)** — falta `INFOSIMPLES_TOKEN`
- [~] **Verificação — Biometria/Face ID (idwall)** — falta `IDWALL_API_KEY` (exige e-mail corporativo + CNPJ)
- [~] **SERVICE_ROLE_KEY** vazia no ambiente — necessária pra verificação server-side e algumas ações admin

C. FALTA CONSTRUIR (estrutura nova — não é só plugar chave)

Pagamento real (a maior peça que falta)

- ☐ **Asaas — integração de verdade** (hoje só existe o webhook esqueleto que retorna 501)
 - ☐ Cliente/SDK Asaas + `ASAAS_API_KEY` / `ASAAS_WEBHOOK_TOKEN`
 - ☐ Carteira pré-paga: recarga via Pix (botão "Adicionar saldo" hoje desativado)
 - ☐ Split automático 80/20 na conclusão da entrega
 - ☐ Subconta do entregador + repasse D+1
 - ☐ Webhook real de confirmação de pagamento (`/api/webhooks/asaas`)
 - ☐ Extrato de carteira pro lojista

Notificações

- ☐ **SMS/WhatsApp** (definir provedor — ex. Zenvia — + credencial + envio do link de rastreio)
- ☐ **Push notifications** (nova corrida pro entregador com som; hoje é polling)

Endpoints pendentes

- ☐ `GET /api/rota` (retorna 501 — proxy de Directions, opcional)

D. JURÍDICO / INFRA / POLIMENTO

- ☐ **Termos de Uso e Privacidade** — hoje são RASCUNHO; validar com advogado (dado sensível de antecedentes = LGPD)
- ☐ Domínio próprio (ex. appdelyvery.com.br) — hoje URL é `appdelivery-psi...` (com "i", a marca é com "y")

- ☐ Limpeza dos dados de teste no banco
 - ☐ Validação visual final no celular (90% dos acessos são mobile)
 - ☐ Definir hospedagem: Impulso hospeda e cobra na mensalidade, ou transfere contas pro dono
-

CAMINHO CRÍTICO pra "operar de verdade"

Em ordem do que destrava o uso real:

1. **Contas no CNPJ do dono** → ligar verificação (3 chaves) — vira o diferencial real do produto
2. **Asaas** → o dinheiro circulando (recarga, split, repasse) — sem isso não fatura
3. **SMS/push** → avisar entregador e cliente (operação fluida)
4. **Jurídico** (termos/privacidade) + **domínio** → pra publicar com cara de empresa séria

Itens 1 e 2 são o coração do "operar". O resto da plataforma já está de pé.

NOTA PRO DOC DE VENDAS (setup R\$15k × Fase 2)

- **Dentro do setup (R\$15k):** plataforma completa (A) + ligar verificação e Asaas (B+parte do C) + termos/domínio = "pronto pra operar".
- **Fase 2 (à parte, evolução):** apps nativos nas lojas (Play/App Store), Van intercidade, conectores Bling/Tiny, múltiplas paradas, agendamento, camada de IA.

Plano de Entrega

Roteiro de trabalho pra entregar a **plataforma pronta pra operar** (o setup contratado). Cruza o que foi **vendido** (plano de negócio) com o **estado real** (auditoria de código+banco, 08/06). Companion do `CHECKLIST-PROGRESSO.md` (esse é o "onde estamos"; este é o "como chegar").

Definition of Done do SETUP (o que o cliente recebe por R\$15k)

A plataforma rodando ponta a ponta com **dinheiro real circulando** e **entregador verificado de verdade**:

1. Lojista recarrega por Pix → cria entrega → entregador aceita → coleta/entrega com comprovante → entregador recebe.
2. Entregador novo passa por antecedentes + CNH + biometria reais antes de rodar.
3. Avisos chegam (entregador sabe da corrida, cliente recebe o link).
4. Termos/privacidade válidos + domínio próprio.

Tudo que está fora disso = **Fase 2 (cobrada à parte)** — lista no fim.

FASE 1 — Ligar a VERIFICAÇÃO (o diferencial) · rápido, destrava valor

Pré-requisito: contas no CNPJ do dono + chaves.

- ☐ **Infra:** criar contas FlagCheck, Infosimples, idwall no CNPJ do dono (idwall exige e-mail corporativo)
- ☐ **Infra:** preencher `.env.local` + Vercel: `FLAGCHECK_API_KEY`, `INFOSIMPLES_TOKEN`, `IDWALL_API_KEY`, `SUPABASE_SERVICE_ROLE_KEY` (hoje vazia — necessária pro server-side)
- ☐ **Backend:** as tomadas já existem (`lib/verificacao/*`, `actions/iniciarVerificacao.ts`) — só validar que rodam com as chaves
- ☐ **Banco/Telas:** já prontos (status de verificação, painel admin de aprovação)
- ☐ **DoD:** cadastrar um entregador de teste → verificação real roda → admin vê resultado → aprova. Provar por `scripts/verify-*`.

FASE 2 — PAGAMENTO real (Asaas) · a maior peça, é o que faz faturar

Hoje só existe o webhook esqueleto (501). Falta construir.

- ☐ **Infra:** conta Asaas no CNPJ do dono + `ASAAS_API_KEY` + `ASAAS_WEBHOOK_TOKEN`
- ☐ **Backend:** client Asaas (`lib/asaas.ts`); criar subconta do entregador no cadastro/aprovação
- ☐ **Backend:** recarga da carteira por Pix (cria cobrança Asaas → credita saldo no retorno do webhook)
- ☐ **Backend:** split 80/20 na conclusão da entrega (entregador/plataforma)
- ☐ **Backend:** repasse D+1 pro Pix do entregador (saque)
- ☐ **Backend:** `/api/webhooks/asaas` real (confirma pagamento → credita; hoje 501)

- ☐ **Banco:** tabelas `pagamentos` / `carteira_transacoes` já existem; revisar se cobrem split/repassse
- ☐ **Telas:** ativar botão "Adicionar saldo" (hoje `disabled`) + extrato da carteira no lojista; tela de saque/ganhos do entregador
- ☐ **DoD:** lojista recarrega Pix de verdade → saldo sobe → entrega desconta → entregador vê crédito → saca. Provar na fonte (read-after-write).

FASE 3 — NOTIFICAÇÕES (SMS/WhatsApp + push)

- ☐ **Decisão:** escolher provedor de SMS/WhatsApp (ex. Zenvia) — credencial no CNPJ do dono
- ☐ **Backend:** enviar **SMS/WhatsApp com o link de rastreo** pro cliente final ao despachar
- ☐ **Backend:** **push** de nova corrida pro entregador (hoje é polling) — Web Push/FCM
- ☐ **Banco:** (se preciso) log de envios
- ☐ **DoD:** cliente recebe o link no celular; entregador recebe alerta de corrida sem estar com o app aberto.

FASE 4 — ENTREGA / polimento final

- ☐ **Jurídico:** termos de uso + privacidade revisados por advogado (dado sensível de antecedentes = LGPD) — hoje são rascunho
- ☐ **Infra:** domínio próprio (ex. appdelivery.com.br) — hoje a URL é `appdelivery-...` (com "i"; a marca é com "y")
- ☐ **Telas:** confirmar que o que já está no banco aparece pro lojista — **retorno** (0026), **cotação de preço** (0025), **proteção de carga** (0027). Banco pronto; validar exposição na UI.
- ☐ **Telas:** validação visual final no celular (90% dos acessos são mobile)
- ☐ **Banco:** rodar `scripts/cleanup-teste.sql` (limpar dados de teste antes de entregar)
- ☐ **DoD:** abrir o app no celular como cliente real, do cadastro à entrega, sem nada de "demo".

AJUSTES POR CAMADA (visão transversal)

CAMADA	SITUAÇÃO	O QUE FALTA
Banco	Muito completo (27 migrations, ~18 tabelas, ~30 RPCs, guards)	Quase nada estrutural — revisar <code>pagamentos</code> / <code>carteira</code> p/ split+repassse; talvez log de notificações
Backend / integrações	Mapbox + Supabase reais; resto é tomada no-op	Construir Asaas (grande), ligar verificação (chaves), criar SMS/push (novo), webhook Asaas real
Telas / UX	3 personas + admin + cliente, todas no ar	Ativar recarga+extrato (depende Asaas); expor retorno/cotação/proteção no lojista; passada final mobile
Infra / jurídico	Deploy contínuo, PWA, SEO ok	Chaves no CNPJ, domínio, termos com advogado, limpeza de teste

FORA DO SETUP — Fase 2 (evolução, cobrar à parte)

Do GAP-FUNCIONALIDADES + roadmap do plano, o que NÃO foi vendido no setup:

- Pagamento **faturado/pós-pago CNPJ** (trunfo p/ conta grande)
- **Agendamento / recorrência** de entregas
- **Múltiplas paradas** (1 coleta → várias entregas)
- **Devolução / return trip + dados do recebedor** (nome/CPF na entrega)
- Instruções de entrega, endereços favoritos, tipos de entrega (express/agrupada)
- **Apps nativos** (Play Store + App Store, via Expo)
- **Van intercidade** (Porto/Gurupi)
- **Conectores** Bling/Tiny/PDV de farmácia
- **Camada de IA** (previsão de demanda, ETA preditivo, antifraude)
- Seguro de carga com **apólice/parceiro real** (hoje só o teto de cobertura no banco)

Polimento do Front, Testes e Vitrine

Auditoria fina (código) de 08/06/2026, tela por tela. O front **funciona** (motores ok), mas está **~70-75% polido**: faltam estados, validações, edge cases e acabamento mobile. Companion do `PLANO-DE-ENTREGA.md`. A validação VISUAL fica com a instância "olhos" (Chrome) + testes em aparelho real.

0. BUGS / GAPS CRÍTICOS (consertar antes de "operar de verdade")

- ☐ **Rastreo mostra entregador FAKE** — `/rastreo/[token]` exibe "Lucas Mendes / Honda CG 160 / ABC-1D23 / 4,9" hardcoded e GPS em simulação (`usePosicaoAoVivo` existe mas não é usado). Tem que puxar do `get_rastreo_publico` real + ligar o GPS ao vivo. **(É o link que foi pro investidor.)**
- ☐ **Dados sensíveis hardcoded no cadastro do entregador** — `defaultValue="Diego Alves de Souza"` + CPF de exemplo no `EntregadorFlow`. Remover.
- ☐ **Cadastro não valida CPF/CNPJ** — aceita "123". Falta máscara + validação (dígito verificador) no negócio e no entregador.
- ☐ **Criar pedido sem saldo não bloqueia** — lojista com carteira zerada cria pedido (entregador entregaria sem garantia de pagamento). Validar saldo antes de criar (pré-flight) — casa com a Fase 2 (Asaas).
- ☐ **Código de entrega sem limite de tentativas** — brute force possível. Throttle (ex. 3 erros → espera).
- ☐ **"Buscando entregador" eterno** — se ninguém aceita, fica travado sem timeout nem opção de cancelar/retentar. Definir timeout + ação.

1. POLIMENTO TRANSVERSAL (vale pra quase todas as telas)

- ☐ **Estados de carregamento** — trocar "Carregando..." por skeletons; padronizar.
- ☐ **Estado de erro com retry** — hoje várias telas falham em silêncio (lista vazia) se a query quebra. Botão "tentar de novo" + toast.
- ☐ **Tempo real nas listas** — Histórico, Ganhos, Comunicados, fila do Admin só atualizam no reload. Ligar Realtime/poll onde faz sentido (ex. comunicado urgente, nova corrida).
- ☐ **Validações de formulário** — máscara e tipo certo: CPF, CNPJ, telefone, valor declarado (`inputMode="numeric"` , sem aceitar valor absurdo).
- ☐ **Mobile fino** — garantir toque $\geq 44\text{px}$ em todos os botões (`.btn` hoje não garante), tabelas com overflow visível, testar **landscape** (painel/mapa apertam), `<pre>` da API e canvas de assinatura sem estourar a tela.
- ☐ **Feedback de ação** — assinatura sem confirmação visual ("assinado ✓"); foto enviada sem check; SlideConfirm sem vibração.

2. POR PERSONA (itens curados, além do transversal)

Negócio

- ☐ Novo pedido: toggle "**retorno**" não tem lógica ligada (existe no banco/0026, falta na tela); endereço fora de Palmas não é barrado; sem timeout no matching.
- ☐ Carteira: botão "Adicionar saldo (em breve)" — ativar com Asaas; tratar saldo baixo/zerado com aviso.
- ☐ Relatórios: mostra 50 na tela mas CSV exporta tudo (alinhar expectativa); avisar export vazio.
- ☐ Perfil: sem edição de dados ainda; conta suspensa não tem aviso/sessão-kill.

Entregador

- ☐ Verificação: animação "aprovado" é cosmética (timer fixo) — tem que refletir o resultado REAL do backend (senão mostra "aprovado" mentindo).
- ☐ Oferta: ao expirar/cair internet no aceite, fica em limbo — precisa re-buscar próxima/reconectar.
- ☐ Coleta/Entrega: geofence 100m pode dar falso-positivo (GPS impreciso em prédio) — mensagem clara de quantos metros faltam; foto/assinatura com validação real.
- ☐ Status "recusado": sem caminho pra reapelar/refazer.
- ☐ Home: ao voltar depois de tempo, pode estar "offline" no backend sem avisar.

Admin

- ☐ PIN de aprovação não é `required` (deixa aprovar sem PIN em alguns pontos) — travar.
- ☐ Comprovantes no drawer de Corridas aparecem vazios quando não há foto — ok, mas confirmar que aparecem quando há (depende da máquina de estados real).
- ☐ Financeiro: texto "movimento real sai quando Asaas existir" — trocar por estado neutro até ligar.
- ☐ Mensagens: sem paginação (carrega todas — trava com volume).
- ☐ Config: sem preview de impacto ao mudar preço; sem histórico de mudança.

Cliente final (rastreo)

- ☐ (ver crítico) dados reais + GPS ao vivo.
- ☐ Token inválido → tela de erro amigável (hoje quebra/branco).
- ☐ Entrega concluída/antiga → mostrar status final claro.

3. SITE VITRINE — conversão (capta cliente?)

Hoje é claro e bonito, mas **não foi feito pra converter**. Falta:

- ☐ **Prova social / números** — "X entregas feitas", "Y negócios atendidos", depoimentos. Hero só tem conceito.
- ☐ **Cobertura no hero** — "Palmas e região" só aparece miúdo no rodapé.
- ☐ **FAQ** — "Como é a verificação?", "Quanto custa?", "Atende meu bairro?" (tira objeção + ajuda SEO/AEO).
- ☐ **CTA do entregador mais forte** — hoje é ghost button discreto; o marketplace precisa dos dois lados.
- ☐ **Tirar "em breve"** da jornada do lojista (sinaliza produto incompleto).
- ☐ **SEO on-page** — title/description/OG por página; schema LocalBusiness/FAQ; imagem de hero real otimizada.

4. PLANO DE TESTES REAIS

Aparelhos (mobile é 90% do uso)

- ☐ iPhone SE / 375px, Android 360px, iPad/tablet 768-1023px (o "vão" tablet é o menos testado), 1 desktop.
- ☐ **2 aparelhos pro GPS** (papel dos "olhos"/Chrome): entregador anda → lojista e cliente veem mover ao vivo.

Cenários ponta a ponta (cada um, na fonte/banco)

- ☐ Lojista cria pedido **sem saldo** → deve bloquear.
 - ☐ Pedido criado → ninguém online → comportamento do "buscando" (timeout?).
 - ☐ Entregador aceita oferta → **perde GPS/internet** no meio → não pode ficar em limbo.
 - ☐ Coleta/entrega **longe do ponto** → geofence avisa; com foto+assinatura+código → conclui e credita.
 - ☐ **Código errado 3x** → throttle.
 - ☐ Rastreo com **token inválido** → erro amigável.
 - ☐ Cadastro com **CPF/CNPJ inválido** → barra.
 - ☐ Admin aprova entregador **sem PIN** → barra.
 - ☐ Suspende negócio/entregador enquanto ele usa → efeito imediato.
-

Prioridade sugerida

1. **Críticos (seção 0)** — principalmente rastreo real + validações de cadastro/saldo (são correção, rápidos e de risco).
2. **Transversal (seção 1)** — estados + mobile fino (eleva tudo de uma vez).
3. **Vitrine (seção 3)** — em paralelo, porque alimenta a captação.
4. **Por persona (seção 2)** — varredura final tela a tela.

Estratégia de Captação

Esboço estratégico (08/06/2026) pra o Eduardo (que é o operador de tráfego) refinar. O APPDELYVERY é um **marketplace de 2 lados**: sem entregador não tem serviço; sem lojista não tem receita. Captar os dois em paralelo, começando pequeno e local.

O desafio do "ovo e a galinha"

Lojista só fica se a entrega sai rápido; entregador só fica se tem corrida. Solução: **começar denso num recorte pequeno** (um bairro/eixo de Palmas + um nicho), não espalhar fino.

LADO A — LOJISTAS (a receita)

Quem (ordem de facilidade × valor)

1. **Âncora**: os negócios do próprio cliente-dono (ele é empresário local) — garante volume inicial real.
2. **Farmácias** (Drogasil usa Bee via API — modelo provado), **óticas**, **lojas de autopeças/eletrônicos**, **mercados/distribuidoras** que entregam no interior.
3. **Vendedores de Mercado Livre / Shopee (Flex)** de Palmas — same-day local (já mapeado no plano de negócio).

Como captar

- **Prospecção direta** (o forte do Eduardo + relação local): visita/WhatsApp com demo no celular (os links que já temos). "Primeiras N entregas grátis pra testar."
- **Oferta de entrada**: isenção de setup / X entregas cortesia / sem fidelidade — reduzir atrito de experimentar.
- **Tráfego local (Google/AEO)**: "entrega Palmas", "motoboy empresa Palmas", "entrega de encomenda Palmas" — o site já tem SEO/llms.txt; falta FAQ + prova social (ver POLIMENTO-FRONT §3).
- **Parceria com sistema/PDV**: virar opção de logística onde o lojista já trabalha (Bling/Tiny têm integração logística) — a API já existe.
- **Indicação lojista→lojista**: comissão/crédito por indicar outro negócio.

Pitch pro lojista (ângulo)

Confiança (entregador de ficha checada) + local (atende rápido, fala com gente daqui) + sem frota própria (paga só o que usar) + comprovação forte (foto/assinatura/código).

LADO B — ENTREGADORES (a oferta/supply)

Quem

Motoboys de Palmas que já rodam pra iFood/99/Bee e querem mais corrida; carro/van pra cargas maiores.

Como captar

- **Grupos de WhatsApp/Telegram de motoboy** de Palmas (canal nº1, baixo custo).
- **Programa de indicação** (já no roadmap): bônus por trazer entregador que é aprovado e roda. Crescimento viral barato.
- **Argumento forte:** recebe **80% do frete**, repasse **D+1** (dinheiro rápido), sem meta punitiva, app no padrão dos grandes.
- **Selo de verificado** como diferencial de orgulho/confiança (e o lojista prefere verificado).

Cuidado de supply

Não captar entregador demais antes de ter corrida (frustra e some). Crescer o supply junto com a demanda do recorte inicial.

SEQUÊNCIA DE LANÇAMENTO (sugerida)

1. **Semente:** 5-10 entregadores verificados + os negócios-âncora do dono rodando de verdade (provar o ciclo no mundo real, não só demo).
2. **Recorte denso:** 1 nicho (ex. farmácias) num eixo de Palmas — virar a referência ali.
3. **Boca a boca + indicação** (os dois lados) + tráfego local ligado.
4. **Expandir nicho a nicho** e bairro a bairro; depois região (Luzimangues) — ver EXPANSÃO-REGIONAL.

MÉTRICAS PRA ACOMPANHAR (desde o dia 1)

- Lojistas ativos / entregas por lojista / recompra.
- Entregadores ativos / corridas por entregador / tempo até aceite.
- Taxa de entrega no prazo, nota média, cancelamentos.
- CAC por lado vs. receita (take rate 20%).

DEPENDE DO PRODUTO (ver POLIMENTO-FRONT + PLANO-DE-ENTREGA)

Captação só escala com: pagamento real (Asaas), verificação ligada (o diferencial do pitch), rastreamento real (não fake), e a vitrine com prova social/FAQ. Captar antes disso = queimar lead.